

ROLE A PLAYBOOK

Data + Photography Score

Pair project · Nathalia & Dafna · Role A

This is not a weather app. Weather is the input. The product is a decision tool for outdoor photographers in Israel.

Photographer asks: I have a portrait in Tel Aviv tomorrow at 17:00. Should I shoot then — and is there a better window?

The product formula

Weather Data → Analysis → Photography Score → Recommendation → Decision

Fill the # TODO (STUDENT) comments. Do not copy a finished weather app. One owner per file.

Your job in one sentence

You turn forecast JSON into a ForecastReport, then into a Photography Score (0–100) and a best shooting window. Role B only displays what you return. She does not parse JSON.

Your files — what each one is for

File	Function (what you write)	Who calls it
photo_planner/models.py	HourlyConditions.from_slot_json — one 3-hour slot. ForecastReport.from_api_json — city + list of hours + sunrise/sunset.	client.py and tests (sample JSON, no internet)
photo_planner/validation.py	normalize_city_name — strip spaces, reject empty. normalize_shoot_type — portrait / sunset / landscape only.	client.py and scoring.py
photo_planner/errors.py	Already written (KEEP). You raise these errors. Do not invent new classes unless you need a new case.	app.py catches WeatherError
photo_planner/client.py	OpenWeatherClient.get_forecast — GET the 5-day / 3-hour forecast. _params — dict with q, appid, units.	app.py after Plan shoot
photo_planner/scoring.py	THE PRODUCT. photography_score, hours_on_date, score_forecast, best_shooting_window. Agree SHOOT_WEIGHTS with Role B on paper first.	app.py · tests/test_scoring.py

tests/	Remove @pytest.mark.skip after each function works. Run pytest from weather-project/.	You, after each step
data/sample_forecast.json	Do not edit. Use it as a map while you write from_slot_json. Date 2026-09-04 has 4 slots (+ 1 extra day to test the filter).	models.py and tests

Your functions — fill these TODOs

Function	Receives	Must return
from_slot_json	one dict from payload["list"]	HourlyConditions (all fields)
from_api_json	full forecast JSON	ForecastReport (city + hours list)
normalize_city_name	whatever the user typed	clean name or InvalidCityError
normalize_shoot_type	"Portrait" etc.	"portrait" or InvalidShootTypeError
get_forecast	city name	ForecastReport, or 401/404 named errors
photography_score	one hour + shoot type	float 0–100
hours_on_date	ForecastReport + "YYYY-MM-DD"	only hours that day
score_forecast	forecast + type + date	list of (hour, score)
best_shooting_window	scored hours	(start, end, score) — MVP = best 3-hour slot

TOGETHER — you both do these (do not split)

These are the only times you must sit together. The rest of the week you can work in parallel.

When	What you do together	Done when
Meeting 1 · start	Decide Role A / Role B (10 min). Copy <code>.env.example</code> → <code>.env</code> , paste the API key. Install with <code>pip install -r requirements.txt</code> .	Each person knows her files. <code>.env</code> exists. Never commit it.
Meeting 2 · notebook	Open <code>notebooks/01_forecast_api_colab.ipynb</code> . GET <code>/data/2.5/forecast</code> for Tel Aviv. Print <code>status_code</code> , then temp, wind, clouds, pop, sunset.	You see 200. Do not open Streamlit yet.
Meeting 3 · weights	On paper: for portrait, sunset, landscape write rain / wind / temperature / clouds weights that add to 1.0. Ask: what would cancel this shoot first?	Both can explain portrait vs sunset. Put numbers in <code>SHOOT_WEIGHTS</code> .
Meeting 4 · plug in	Connect <code>app.py</code> to real <code>get_forecast</code> + <code>score_forecast</code> . Try Tel Aviv portrait tomorrow 17:00. Try an error case.	The page shows a Photography Score. Mistakes show <code>st.error</code> , not a traceback.
Meeting 5 · ship	GitHub without secrets (<code>docs/DEPLOY.md</code>). Streamlit Cloud, secret name <code>OPENWEATHER_API_KEY</code> . Screenshots of score + error.	Public URL works. <code>.env</code> is not on GitHub.

You will NOT wait all week

Field names and function names are already in the starter files. Role A fills how they work. Role B can already use the names. The only way to block each other is changing a field name in `models.py` without saying so.

Your solo path (Role B is not blocked)

You do not need Streamlit. Sample JSON + pytest are enough until Meeting 4.

Order — one file at a time

1. Notebook with Role B until 200.
2. `models.py` using `sample_forecast.json` (freeze field names — they are already KEEP).
3. `validation.py`
4. `client.py` `get_forecast` for Tel Aviv
5. Meeting 3: weights on paper with Role B
6. `scoring.py` (the product)
7. Turn tests on, run `pytest`
8. Meeting 4: help Role B plug `app.py` into your functions

Contract — do not rename these fields

Role B already builds charts from these names. If the UI needs a new field, add it here first and tell her.

Object	Fields Role B will read
--------	-------------------------

HourlyConditions	time_text, timestamp_unix, temperature_c, feels_like_c, humidity, wind_speed, cloud_cover, rain_probability, rain_mm, description, icon
ForecastReport	city, country, latitude, longitude, timezone_offset_seconds, sunrise_unix, sunset_unix, hours

Hints: pop is 0–1 (0.4 = 40% rain). The key "3h" is a string. rain is often missing — use .get. Endpoint is /data/2.5/forecast, not current weather.

Your checklist

□ API key only on your computer
Create key → copy .env.example to .env → paste. Never commit .env.

□ Read one raw forecast slot
After 200, print city name, len(list), temp, pop, clouds.

□ Parse sample JSON in models.py
from_slot_json + from_api_json with data/sample_forecast.json.

□ Validate city and shoot type
Both functions in validation.py.

□ Live request in client.py
Clean name, requests.get, return ForecastReport.from_api_json(...).

□ Map HTTP failures to errors
401 InvalidApiKeyError · 404 CityNotFoundError · else WeatherRequestError.

□ Filter hours to the booked date
hours_on_date — 2026-09-04 keeps 4 slots.

□ Write photography_score
Each weather field → 0–100 goodness, then SHOOT_WEIGHTS.

□ score_forecast + best_shooting_window
MVP = the single best 3-hour slot.

□ pytest passes
Delete @pytest.mark.skip as you go. Run pytest from weather-project/.

Rules

Do not commit secrets. Do not copy a weather app. Do not download a “perfect” score formula — you and Role B decide the weights. Fill TODOs yourself.